CLASS 2

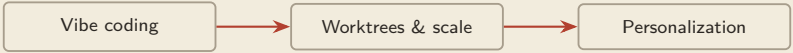
Claude Code at *Research Scale*

Vibe coding, worktrees, and personalization.

Jiaqi Shao · NUS Business School

Roadmap: Three Ways to Work at Scale

You already direct one agent — now direct many, and shape them to your research.



- Part 1 — Vibe coding. Delegate the typing, keep the judgement: contracts, plan mode, memory, tests.
- Part 2 — Worktrees & scale. Many sessions in parallel, without collisions.
- Part 3 — Personalization. Subagents, skills, MCP, hooks, GitHub.

3 parts · 3 exercises · 3 quizzes · 2 breaks.

NUS BUSINESS SCHOOL

Before We Start: A Show of Hands

Four topics, one quick scan — so I can pace today to this room.

Topic	Where it lives
Claude Code	all three parts today
MCP	Part 3 — connecting data sources
Agents & subagents	Parts 2–3 — working in parallel
The LLM wiki	Class 3A — where today is heading

fist = never touched it · **1 finger** = tried it once or twice · **2 fingers** = use it regularly

NUS BUSINESS SCHOOL

Three Ideas That Run Through Everything

The same three messages as Class 1 — today each one grows.

	The message	Today it becomes
1	**Agentic partner** — a junior collaborator that plans, acts, reflects	a *team* of collaborators you orchestrate
2	**Context engineering** — context quality caps output quality	engineered at scale: scoped sessions, lean memory
3	**Persistent memory** — `CLAUDE.md` as the project's second brain	built out in full; Class 3A grows it into a knowledge system

What We Carry In: Three Risks, One Machine

Shirley's three failure modes don't go away at scale — they multiply.

- **Hallucination** — confidently wrong output.
- **Sycophancy** — agrees with you.
- **Automation bias** — you over-trust the machine.

more agents, more actions → less direct observation of each step
(delegation risk)

model proposes the next step · tools change the project · harness connects both inside a controlled loop

the thread — everything today is about keeping the evidence chain intact while the machine speeds up

Recall: The Junior Collaborator

Before we build on it, pull it back from Class 1B — from memory, not from the slide.

Class 1B said Claude Code is a **junior collaborator**, not a code generator.

Yet the model itself only ever predicts the next word.

What exactly turns that model into a collaborator that plans, acts, and checks its work?

Think first — 30 seconds — answer on the next slide, no peeking.

NUS BUSINESS SCHOOL

The Model Was Never the Bottleneck

The answer: the harness — everything in the system EXCEPT the model weights.

The model	The harness
predicts a useful next step	connects tools, context, guardrails, and verification in a loop
intelligence inside one call	the surrounding system that turns a call into controlled action

- The evidence: hold a *deliberately weak* model fixed, never improve the prompt — engineer only the harness, and task completion still becomes reliable.

Capability lives in the system, not just in the weights.

harness — everything except the model weights — and it is the part YOU can engineer

The Operating Principle

Once the harness is the lever, an agent mistake stops being a scolding matter.

> "When the agent makes a mistake, engineer the environment so it can't make that mistake again."
> — M. Hashimoto

- The fix is rarely "try harder" — it is a *missing harness capability*.
- **Guardrails** block structural failures **before** the call runs; **verification** reads the execution trace **after** — the trace can't lie about which tools fired.
- Part 3 builds exactly these: **hooks** are the guardrails; **referee subagents** are the verification.

harness engineering — everything today — plan mode, `CLAUDE.md`, worktrees, subagents, hooks — is this one discipline

Same Model, Different Harness, Different Machine

One worked contrast: the same-class model, dropped into three harnesses.

The harness around the model	What the machine can now do
bare chat window — no tools, just weights	answers from memory; cannot open a file, cannot check a claim
+ web-search tool + a cite-and-check loop	an "AI search" product: fetches live sources, quotes them, verifies against them
+ terminal, file tools, and the full loop	Claude Code: reads your project, edits, runs, tests — a working collaborator

- Same "IQ" in every row — the model never changed. Each row only *adds harness*.

The capability difference between AI products is mostly a harness difference.

the lesson — when you compare AI tools, you are comparing harnesses — so learn to read them as harnesses

NUS BUSINESS SCHOOL

The Market Has Voted: The Harness Is the Product

As of mid-2026, every serious vendor ships a harness — not just a model.

Vendor	The harness	Its shape
Anthropic	Claude Code	terminal + file tools around Claude models — today's classroom machine
OpenAI	ChatGPT agent mode	a browse-and-act harness around GPT models
Moonshot	Kimi Code CLI	open-source (Apache-2.0) harness around the open-weight Kimi K2 family; built to sustain hundreds of sequential tool calls

- Models are converging; **harnesses are where products now differentiate** — the harness is the competitive layer.

Vendor line-up as of mid-2026 — this landscape moves fast; verify vs current announcements.

why you care — the durable skill is not any one product — it is harness engineering, and it transfers

An Economist's Harness, End to End

So far the harness is an idea. Here is a working one, published by an economist — read it as a parts list.

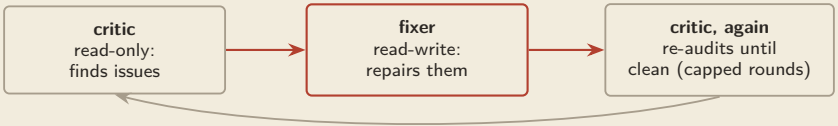
His files	The harness component
`CLAUDE.md` — a ∼120-line "constitution": short directives + pointers, nothing more	**context / memory** — kept slim because the model follows only so many standing instructions
path-scoped rules in `.claude/rules/` — R conventions load only when an `.R` file is touched	**context, on demand** — expertise that costs nothing when irrelevant
skills: `/compile-latex`, `/proofread`, `/deploy`	**procedures** — written once, invoked forever

P. Sant'Anna, public workflow guide, 2026 (paraphrased) — a full Claude Code setup for course development and research.

look closer — every row is a concept from this course, running in production for economics teaching and papers

Inside His Verification Layer: Critic + Fixer

The adversarial pattern: the author never gets to approve the author.



- The critic *cannot* fix files, so it has no incentive to downplay issues; the fixer *cannot* approve itself — the critic re-audits.
- An orchestrator "contractor mode" runs this loop and scores the output against **quality gates**: **80** to commit · **90** for a PR · **95** aspirational.

verification, engineered — "looks good to me" is replaced by a loop the author cannot sweet-talk

His Memory and Guarantees — and the Point

The last two component groups: memory that survives, and rules that fire no matter what.

- **Plan-first** — every approved plan is saved *to disk*, so strategy survives when the context window is compressed or the session ends.
- **Session logs** — record *why* decisions were made, not just what changed; a future session (or a coauthor) can reconstruct the reasoning.
- **Hooks** — enforce the rest: block destructive git commands, save state before compaction, auto-write the logs the model would forget.

None of this required training a model — it is all harness, and it is all plain files you can copy.

P. Sant'Anna, public workflow guide, 2026 — psantanna.com.

the arc of today — by the wrap-up you will have built a small version of exactly this system

PART ONE

Vibe Coding

- *Delegate the typing; keep the judgement.*

What "Vibe Coding" Really Means

You describe intent and steer; the agent handles the keystrokes.

You own	The agent handles
the research question	syntax and boilerplate
what "correct" means	running and re-running
verifying the result	searching the codebase

- "Vibe coding" sounds casual — done properly it is the *most disciplined* way to work: you delegate the keystrokes, never the judgement.
- That's the deal on the table: intent and standards stay with you; syntax, execution, and search go to the agent.

the deal — describe intent and steer; the agent handles the keystrokes — but nothing on your side may be delegated

Every Column on Your Side Is Judgement

What stays with you is never typing — it is judgement.

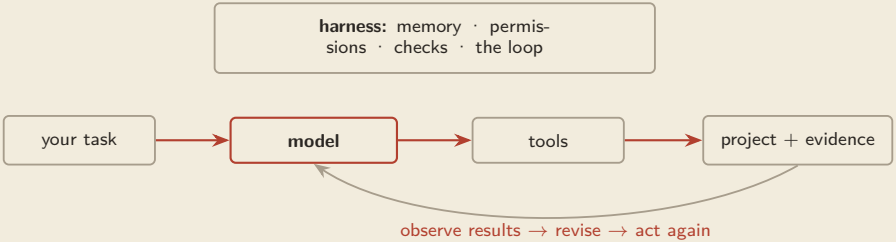
the research question	→	*what's worth asking*
what "correct" means	→	*what evidence would satisfy a referee*
verifying the result	→	*whether this output earns trust*

Because generation is fast, the scarce discipline is reading and checking — not producing.

speed — only a gift if you still verify — automation bias is the tax on unchecked speed

What Turns a Language Model into an Agent?

Not a larger brain alone — a system that lets it act, observe, and try again.



Class 1 used this loop. Class 2 configures what it remembers, may do, and must prove.

Part 1 Toolkit: Four Disciplines

Four structures that keep judgement in the loop as speed goes up.

- Each is taught the same way: first the *failure it prevents*, then the practice, then why it matters.
- They compound: the contract feeds the plan, the plan's standing rules persist in memory, and tests check what the contract promised.

For each: problem → practice → meaning.

What Must the Agent See to Stop Guessing?

A request fails when the missing context forces silent choices.

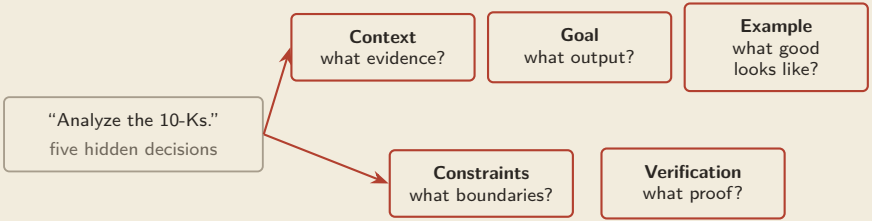
Context quality caps output quality.

- You decide what the model *sees* — files, facts, constraints.
- Everything you don't provide, it must *guess*.
- Vague in, vague out — the model isn't weak; you made it guess.

context — everything in the model's window right now: your words, the files it read, the history of this session

Why Can "Analyze the 10-Ks" Produce Five Answers?

Because the agent must guess the input, output, standard, boundary, and proof.



A prompt contract moves those decisions from the agent back to the researcher.

Worked Contract: 10-K Risk Factors

The full contract on a real filings task — students photograph this slide.

Context: `@filings/aapl-10k.txt` is a raw 10-K.

Goal: extract Item 1A Risk Factors into a table.

Example: one row = (risk title, one-line summary).

Constraints: keep only Item 1A; plain text, no HTML; never modify files in `filings/` — write output to `out/`.

Verification: report how many risk factors were found, and flag any filing section you could not parse — **do not silently skip**.

`@` — hands the agent an exact file — pointing beats pasting, pasting beats describing

Why Constraints Beat Step Lists

Constraints draw the boundary; step lists grab the steering wheel.

✓ "Keep only Item 1A; never touch `filings/`; flag failures."

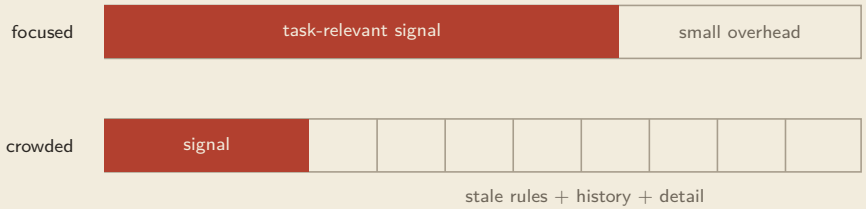
× "First open the file with pandas, then regex for 'Item 1A', then loop over…"

The agent's power is the loop — plan, act, observe, re-plan. A step list freezes its plan at **your** *first guess — and your first guess about code is the weakest link in the room.*

division of labour — you define what "correct" means; it finds the path

Why Can More Instructions Make the Result Worse?

The working context is finite; irrelevant rules compete with the rule that matters now.



Keep always-on rules short; load specialist detail only when the task needs it.

one principle, used all day — why charters stay short, skills load on demand, and sessions get scoped

NUS BUSINESS SCHOOL

Prompt Clinic 1: The Vague Ask

The most common failure: making it guess.

× "Analyze the filings data."

forced guesses: which files? · what question? · what output? · what counts as done?

✓ "Using `@out/filings.db`, count filings per fiscal year 2020–2024 and output a two-column table (year, count). I expect roughly 120 total."

golden rule — show your prompt to a colleague with no context; if they'd be confused, so is the model

Prompt Clinic 2: The Micromanaged Script

The opposite failure: doing its job badly instead of yours well.

× "Open the CSV with pandas, rename column 6 to 'ret', then merge with how='inner' on permno, then…"

steps frozen at your first guess · no room to adapt · the step list is itself probably wrong

✓ "Merge the two files into a firm-year panel. Constraints: keep 2010–2023, don't modify raw files. Verification: report the match rate; I expect ∼4,500 firms."

the tell — dictating code you couldn't write? stop — state the boundary and the acceptance test instead

Point at Files with `@`

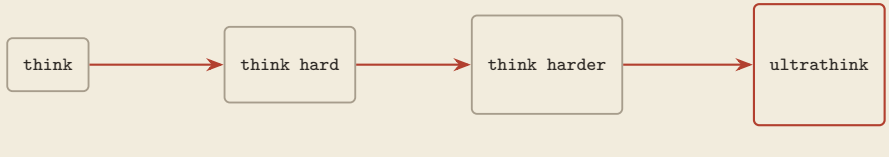
The cheapest context-engineering move: hand it the exact file.

- Type `@` + first letters → picker appears → that exact file becomes context.
- Works for folders too: `@filings/`.
- What actually happens: the file's real content enters the context window — the agent works from the artifact itself, not from your description of it.
- Rule of thumb: **pointing beats pasting, and pasting beats describing.**

the difference — "the file I mentioned earlier" is a guess; `@filings/aapl-10k.txt` is a fact

Ask for More Thinking, by Name

Four literal keywords raise the reasoning budget.



- These are **literal words you type in the prompt** — not menu commands, not settings.
- Each rung buys a larger *internal reasoning budget* before it acts: more alternatives explored, more self-checking — and more time and cost.

Keyword ladder per the DLAI course; wording can evolve — verify vs current docs.

The Thinking Cost Model

More thinking costs time and money — spend it where reasoning is the bottleneck.

Worth it	Not worth it
architecture decisions	routine edits
gnarly bugs	renames
complex refactors	formatting
research-design trade-offs	anything a plain tool does

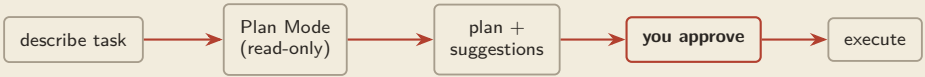
It's telling a colleague "take a proper look at this one" — you don't say that about every email.

reasoning budget — a resource like RA hours: allocate it to the hard problems

How Do You Inspect the Plan Before Anything Changes?

The need: let the agent investigate freely without granting permission to edit.

Can	Cannot
read files	write or edit files
search the project	run state-changing commands
reason and draft a plan	commit



toggle: `Shift+Tab` · nothing is touched before approval

why it works — reading is how it builds a good plan; the write-lock is what makes reviewing it free

Plan Mode in Practice

Think, then plan, then execute — a free architectural review.

- Best for **new features, complex bug fixes, refactors** — anything with more than one defensible design.
- Read the plan like a referee: right files? right steps? anything *silently dropped*?
- To redirect: don't approve — steer, and it re-plans.

In the demo, watch for the one visible behaviour: the plan arrives and the agent *stops*. Nothing was edited.

that pause — is where your judgement lives — the review costs minutes; an unreviewed plan can cost the dataset

Three Design-then-Code Paradigms

Decide the workflow before the first line of code.

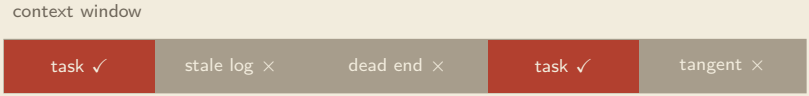
Paradigm	Note
explore → plan → code → commit	general default
write tests, commit → code, iterate, commit	**TDD — strongest**
write code → screenshot → iterate	treats symptoms — weakest

- Why the middle row wins: a test is a *self-checkable* target — the agent runs it, sees red, and iterates to green without you re-explaining what "done" means.

Tests give the agent a target it can check itself against.

Why Long Sessions Decay

Everything ever said in a session stays in the window — including the dead ends.

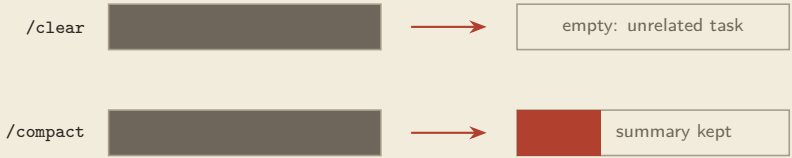


- The window carries your *whole* history — failed attempts, stale logs, tangents.
- Finite attention (the budget from a moment ago) now spreads over noise, so quality and focus drop.

the failure — a cluttered context makes the agent slower and dumber — not because the model changed, but because you diluted it

/clear vs /compact: The Decision Rule

Reset on purpose: wipe for a new topic, compress for a long one.



- **New topic** → /clear.
- **Same topic, getting long** → /compact.
- **Constraint you can't afford to lose** → write it into CLAUDE.md — don't trust the summary.

/compact — keeps a summary, not every detail — durable rules belong in memory

NUS BUSINESS SCHOOL

Recall: The Second Brain

One more carry-over from Class 1B — reconstruct it before we rebuild it.

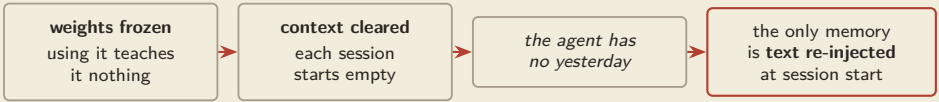
Class 1B called `CLAUDE.md` the project's **second brain**.

What did that mean — and what physical facts about the model FORCE that brain to be external?

Two facts, one consequence — answer on the next slide, no peeking.

The Problem: Total Amnesia Between Sessions

Physical fact → consequence → practice: why memory must be a file.



Recall answer: the second brain = external, versioned, shared text — re-injected every session.

So: put standing knowledge in a file it auto-reads. Class 3A grows this file into a full research wiki.

course message three — persistent memory has to be engineered; it never happens by itself

What Must the Next Session Remember Automatically?

Repeated corrections belong in a versioned project charter, not in your memory.

Only standing knowledge

- how to run and test the project
- where evidence and outputs live
- what must never be changed

CLAUDE.md

- Use `uv` to run the server.
- Data in `data/`; **never edit raw**.
- Run `pytest` before commit.
- Sample period 2010–2023.

Commit it to git — every teammate and every future session inherits the same rules.

If a rule is not needed every session, keep it out of the charter.

Prompt vs `CLAUDE.md`

A prompt is one-off; the charter is permanent and shared.

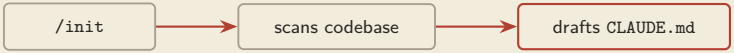
Prompt	`CLAUDE.md`
this session only	every session, automatic
you must remember to say it	written once
yours alone	shared via git with team + future you

- The tell: typed the same instruction three times this week? It's a charter line, not a prompt line.

Say it once → prompt; need it always → charter.

Bootstrap with `/init`

Let the agent read the repo and draft its own charter.

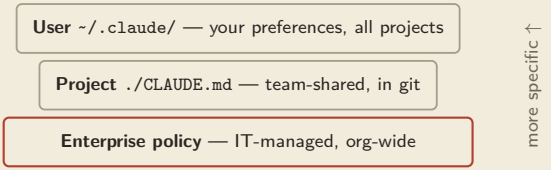


- Read the draft — it's a starting point, not gospel.
- Trim what's wrong, add what it missed.
- Commit, then refine as the project grows.

the draft — treat it like RA work: useful first pass, your judgement finishes it

Memory Comes in Layers

Broad organization-wide rules at the bottom; specific project and personal rules on top.

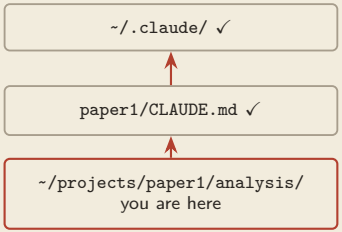


- More specific layers *supplement or override* broader ones.

Exact precedence order — verify vs current docs.

How Memory Is Found: Search Upward

From wherever you launch, it walks up the folder tree collecting `CLAUDE.md` files.



- Launch anywhere inside the project — the charter is still found.
- `@path/to/file.md` *imports* other files into a charter — this replaced the deprecated `CLAUDE.local.md`, and plays well with the worktrees of Part 2.

Import syntax & deprecation status — verify vs current docs.

memory follows the folder tree — which is why folder structure is part of context engineering

Save a Memory on the Fly with

*Notice a rule mid-session? Prepend **#** and it becomes permanent.*

```
# Always use uv to run the server, do not use pip directly.
```

- It asks which memory file to write to (project or user).
- A one-off correction becomes a durable convention.
- The agent stops repeating the mistake — small habit, compounding payoff.

Why Tests: The Agent Can Claim Anything

Problem first: "it works now" is a claim, not evidence — and this model hallucinates claims.

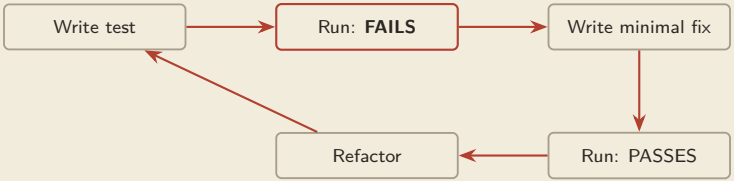
- The agent reports success in the same confident voice whether it's right or wrong *(hallucination — Class 1A)*.
- Your read of the diff can be sweet-talked; a **failing-then-passing test cannot**.

A test that never failed proves nothing.

red then green — the test must fail on the broken code first — that failure is what makes the later pass meaningful

Test-Driven Debugging: Red then Green

Don't patch blindly — write a test that reliably reproduces the bug.



- The first **FAILS** is the evidence that the test actually tests something — only then does the later pass mean anything.
- At the end, two things are true: the bug is fixed, and the repo has permanent immunity — if the bug sneaks back, the test screams.

keep the test — the passing test stays in the repo — an immune system against this bug's return

Worked Example: The RAG Bug — the Setup

The mindset flip: the instruction was NOT "fix the bug."

The symptom

- a RAG chatbot answers "query failed" to a content question

The actual instruction:

"Write tests for the core components, run them against the current system to identify which component is failing, and propose fixes based on what the tests reveal."

Ask it to *locate via tests*, not to *guess a fix*.

RAG — retrieval-augmented generation — fetch relevant passages first, then write an answer grounded in them

Worked Example: The Diagnosis

Tests turned a vague symptom into an exact address.

The agent's plan

- create `tests/`
- unit + integration tests for the generator, the RAG system, the search tools
- mock the database
- run the suite

Root cause: `MAX_RESULTS = 0`

a config value — the search was asking for zero results. Fix the config → all green.

"query failed" (anywhere in ∼10 files) → one misconfigured constant (one line).

the tests — didn't just find the bug — they proved the fix, and they remain as guards

A Reusable Test-First Prompt

Copy the shape; swap in your files and your symptom.

```
The RAG chatbot returns 'query failed'. I need you to:
1. Write tests for the core components in @backend/search_tools.py,
  @backend/ai_generator.py, and the RAG system itself.
2. Save the tests in a tests/ directory within @backend.
3. Run these tests to identify the failing component.
4. Propose a fix based on the test results.
Think.
```

The closing **`Think.`** buys extra reasoning before it writes a single test (the thinking ladder).

Quiz 1 — Approve This Plan?

A real Plan-Mode plan is waiting for your approval — read it like a referee.

Plan for "build the EDGAR filings database":

1. Design SQLite schema (cik, company, fiscal_year, item, text).
2. Write parser tests first.
3. Parse `filings/` into the DB.
4. Skip any filing that fails to parse, and delete malformed files from `filings/` to keep the folder clean.
5. Report row counts.

Approve / approve with changes / reject — and which line made you decide?

Vote in 20 seconds — answer on the next slide, no peeking.

Quiz 1 — Answer

Step 4 fails twice — and "skip silently" is the researcher's tell.

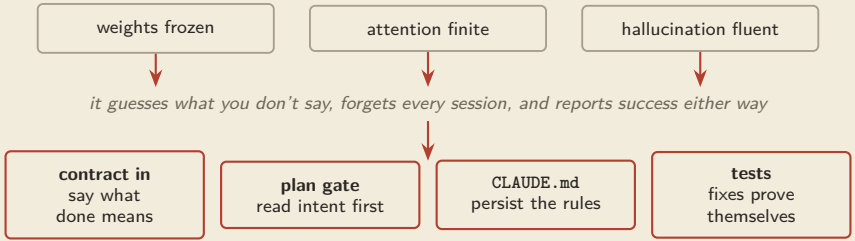
- × ~~"skip any filing that fails to parse"~~ — silent sample loss → selection bias you'll never see.
- × ~~"delete malformed files"~~ — destroys raw data, your read-only zone.

✓ "Approve steps 1–3 and 5. For step 4: never delete or silently skip — log every failed filing to `failed_filings.txt` with the reason, and leave `filings/` untouched."

the reflex — plans sound sensible in bulk; failures hide in single lines — read line by line, and make every drop loud

Part 1 Conclusion: The Derivation Chain

Why these practices are necessary, not stylistic.



"Never type a task — type a contract; and let nothing claim 'fixed' without a red-then-green test."

all four disciplines are harness engineering — when the agent errs, fix the environment, not the mood

Exercise 1 — Watch the First Move

I run the opening 90 seconds; then it's yours.

- 1 `Shift+Tab` into Plan Mode.
- 2 Paste the schema contract (next slide).
- 3 The plan arrives — *we read it together before anyone approves.*

Exercise 1 — Brief: A Structured EDGAR Database

Plan Mode to design it; tests to make it correct. **Goal:** parse a folder of 10-K filings into a queryable SQLite database, test-first.

```
In Plan Mode: design a SQLite schema for 10-K filings
(cik, company, fiscal_year, item, text). Show the plan first.
Then write tests that parse @filings/ into the DB before writing
the parser. Log unparseable filings to failed_filings.txt ---
never skip silently, never modify filings/.
```

No SQL needed — describe the table in English and check the schema makes sense.

Agent skipped the plan? `Shift+Tab`, then "show me the plan before writing anything"; `Esc` re-steers.

Exercise 1 — What Good Looks Like

You approved a plan before any file changed — and a test failed before it passed.

Good signs

- plan reviewed in read-only mode
- a test reproduced the parsing gap *first*
- schema normalized — one row per item
- failed filings logged, `filings/` untouched

How to verify

- spot-check one filing by hand
- re-run tests — all green
- ask "count filings per fiscal year" — total matches the folder?
- `cat failed_filings.txt` — every miss accounted for

If you can't check the answer, you don't yet have one — steer and re-run.

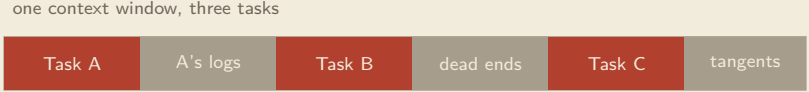
PART TWO

Worktrees & Scale

- *One repo, many sessions — without collisions.*

The Scale Problem: One Session, Many Tasks

The tempting move — pile every task into one long session — quietly fails.



- Each task's instructions and leftovers dilute attention on every other task (the attention budget, third harvest).
- Unrelated context isn't neutral — it actively degrades adherence to your key rules.

the failure is invisible — no error message, just gradually worse work

The Scoping Rule

Derived, not decreed: one task, one session, one clean context.

finite attention → unrelated context degrades all tasks →
independent tasks get separate, tightly-scoped sessions.

- × one bloated session doing three things
- ✓ three lean sessions doing one thing each

scoping — context engineering at the session level — message two, scaled up

Scale Is Delegation Risk, Managed

Class 1A said it: more autonomy means answerability must be engineered back in. The risk: several agents acting at once → you observe *less* of each action *(delegation risk)*.

The three engineered answers

- **Isolation** — worktrees: agents physically can't touch each other's files.
- **Standard procedure** — commands: the task brief is written once, reviewed, reused.
- **One audit point** — the merge: every change funnels through a step you review.

the deal — we scale the agents AND the audit trail together — never one without the other

How Do You Stop Rewriting the Same Task Brief?

Repeated prompts drift; a saved command makes the procedure reviewable and reusable.

Without a command

"Clean this filing."
"Also log failures."
"And never edit raw."
→ wording drifts each run

With a command

`/clean-filing <file>`
scope + constraints + audit output
→ one reviewed brief, every run

A custom command is a versioned RA instruction sheet you only write once.

Anatomy of a Good Command

Bake the constraints into the file, not into your memory.

implement-feature.md

Implement: `$ARGUMENTS`

IMPORTANT: front-end only.
Write your changes to
`frontend-changes.md` —
assume you may modify that file.

- scope constraint — "front-end only"
- audit artifact — "write your changes to `frontend-changes.md`"
- standing permission — "assume you may modify that file"

Whitelist any permissions the command needs in `.claude/settings.local.json`.

the pattern — everything you'd otherwise have to remember to say — the command says every time

Commands vs `CLAUDE.md`

One is always on; one runs only when called.

	`CLAUDE.md`	`.claude/commands/`
When	every session, automatic	only when invoked
Role	standing background rules	scenario-specific procedure
Analogy	the house style	a saved recipe

Always-true facts → memory; repeatable procedures → commands.

the split — protects the attention budget — the charter stays short because recipes live elsewhere

Commands You'll Actually Write

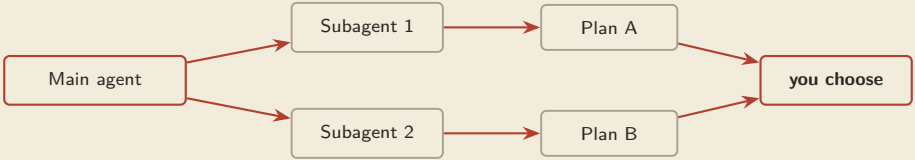
Three research recipes worth saving this week.

- `/clean-filing <file>` — strip HTML, verify Item structure, log anomalies.
- `/ref-check <draft>` — verify every citation in a draft resolves and is quoted accurately.
- `/run-pipeline <groups>` — the Exercise-2 command: worktree per group, clean → extract → summarize, write to `out/`.

If you've typed the same multi-line instruction twice, it wants to be a command.

How Do You Get Rival Plans Before Committing?

Ask for competing plans before committing to one.




```
Use two parallel subagents to brainstorm possible plans.
Do not implement any code.
```

The last clause is load-bearing: you want options to compare, not code to clean up.

subagent — a helper with its own separate context — temporary today; permanent specialists in Part 3

Worked Example: Two Plans Come Back

Refactor the generator to allow two sequential tool-call rounds — two subagents, two answers.

Plan A — iterative	Plan B — redesign
loop inside the existing method	new helper methods
smaller, safer change	cleaner long-term
quick to verify	bigger, riskier change

the user's reply:

```
Implement approach A.
```

The Subagents Advise; You Commit

The decision between a safe small change and a clean big one is exactly the judgement that stays with you.

- Two *independent* contexts → genuinely different options, not an echo of your first idea *(counter-sycophancy by construction)*.
- The trade-off — risk now vs cleanliness later — is a research-taste call, not a coding call.

They advise; you commit.

why two — asking one agent "is my plan good?" invites agreement; asking two agents for rival plans forces a real comparison

The Collision Problem

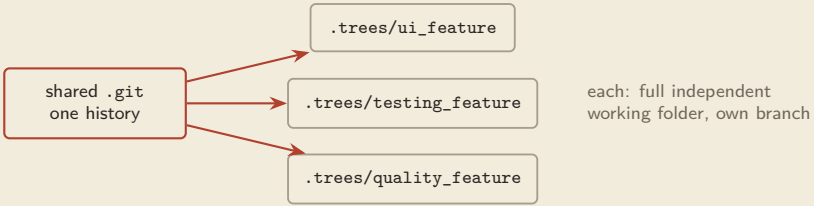
Two sessions, one folder: the second writer wins, silently.

- Whoever saves second silently destroys whoever saved first — the file simply *is* the second version afterwards.
- Same problem as two humans on one shared folder — this is why version control exists, and git has a purpose-built answer.

isolation — must be physical: separate working copies, not separate chat windows

How Can Two Agents Edit Without Overwriting Each Other?

Give each task a separate working folder and branch, while keeping one shared history.



- Each folder is a *complete, independent* workspace on its own branch — edit, run, test in one without touching the others.
- An agent session in folder one physically **cannot** overwrite files in folder two — the isolation Part 2 promised, delivered by git itself.

worktree — extra working copy wired to the same repository — the history is shared; the files are not

Clone vs Worktree

A clone copies and disconnects; a worktree shares and stays in sync.

`git clone` ×3	`git worktree` ×3
three full copies of everything	one history, three working folders
three *disconnected* histories	branches merge natively — one repo
merging = pushing between repos	merging = ordinary branch merge
heavy on disk	light: no duplicate history

Worktrees give clone-level isolation at branch-level cost.

think — three desks in one office sharing one filing cabinet — not three separate offices

What `git worktree add` Actually Does

One command, two effects: a new folder and a new branch.

```
git worktree add .trees/ui_feature
```

- → creates folder `.trees/ui_feature` with a full checkout.
- → creates branch `ui_feature` **off your current HEAD** — wherever you are now, not magically "off main".

So: run your adds from the commit you want as the common base — usually an up-to-date `main`.

the branch — is the worktree's identity — the merge later is just ordinary branch merging

Set Up the `.trees/` Workspaces

Three commands, three parallel workspaces.

```
git worktree add .trees/ui_feature
git worktree add .trees/testing_feature
git worktree add .trees/quality_feature
```

```
git worktree list   # shows main + the three trees
```

- Run all three adds while standing on an up-to-date `main` — every tree then shares a clean common base.

Commit or stash your current work first — a dirty tree is the classic `add` failure.

NUS BUSINESS SCHOOL

Housekeeping: Ignore, Open, Launch

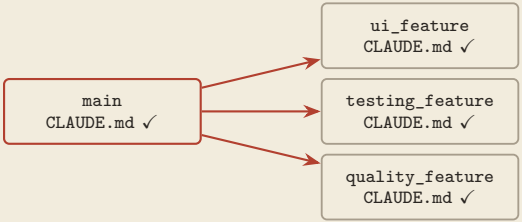
Three habits that keep parallel work tidy.

- **1** Add `.trees` to `.gitignore` — working views must not be committed.
- **2** Open each worktree folder in its own terminal.
- **3** Start a separate Claude Code session in each — *one task*, *one session*, *one worktree*.

the payoff — the scoping rule now has a physical address

Memory Travels with the Worktree

Each worktree contains the project's `CLAUDE.md` — every parallel agent reads the same charter.



a committed file is in every checkout — your rules govern all parallel sessions

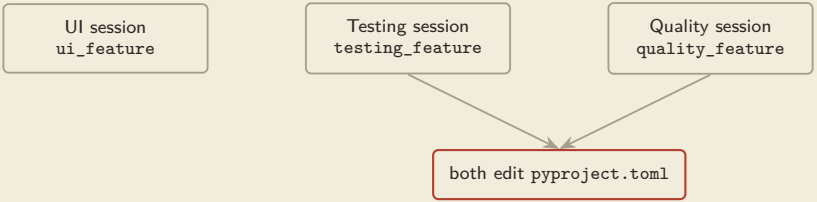
`@path` imports (the memory-search slide) resolve inside each worktree — why imports replaced the old local file.

Import mechanics — verify vs current docs.

write once — the rule is written once; five parallel agents obey it

Three Tasks, Running in Parallel

Three sessions live at once — UI, testing, quality — each in its own folder.



- Each session starts with a *clean context scoped to one task* — the scoping rule and the isolation rule working together.
- You rotate between terminals like a PI checking on three RAs — steer one while the others keep working.

Two of them are about to edit the same file…

The Conflict Is Deferred, Not Lost

In separate worktrees, a same-file edit is a decision postponed — not work destroyed.

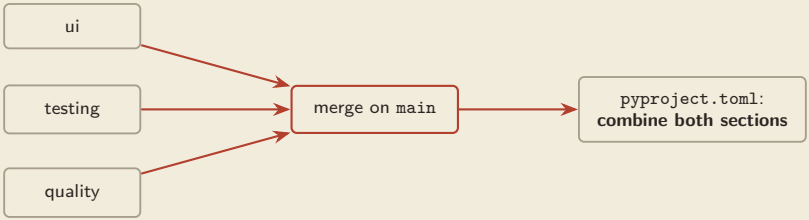
- Each session edits *its own copy* on *its own branch* — nothing is overwritten.
- The disagreement becomes a **merge conflict later**: visible, reviewable, resolvable.

shared folder	worktrees
second save silently wins	both versions preserved; conflict surfaces at merge

the trade — isolation buys safety now and a clean decision later

Merge and Resolve, Back on `main`

Return to the main folder and let the agent stitch the branches together.




```
Use git merge to merge all the worktrees in .trees/ into main
and fix any conflicts.
```

- One instruction from the main folder stitches all three branches back — conflicts surface *here*, in one place, instead of silently during the work.

The pytest-config vs formatting-config clash is resolved by keeping both — not by picking one.

Review the Merge Like Any Draft

The agent's conflict resolution is a proposal — the audit point is yours.

- Open the merged file — are *both* sections actually intact?
- Run the tests on merged `main` — green?
- `git log --oneline` — every branch's commits present?

"It merged cleanly" is a claim; the three checks are the evidence *(automation bias, again).*

Part 2 promised one audit point — this is it — don't wave it through

NUS BUSINESS SCHOOL

Cleanup: Leave the Repo Ready to Go Again

After the merge: remove the views, keep the history.

- Ask the agent to remove the `.trees/` worktrees and their merged branches (or: `git worktree remove <path>`, `git branch -d <name>`).
- Confirm with `git worktree list` — only `main` remains.
- `.trees` stays in `.gitignore` for next time.

worktrees — disposable scaffolding; the merge made their work permanent

When to Parallelize: The Decision Rule

Parallelism pays only when tasks are independent.

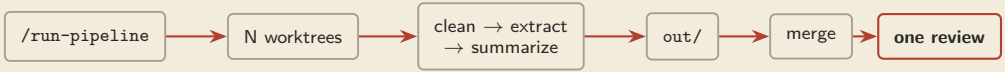
- Tasks **independent** (touch different concerns) → worktrees + parallel sessions.
- Tasks **dependent** (each needs the last one's output) → one session, sequential — parallel would just guess at inputs.
- Task **mechanical** → no agent at all (Class 1B rule).

Start with 2–3 parallel sessions to prove the flow; scale after the merge comes back clean.

the pattern — isolation-plus-merge — the number of agents is the boring part

The Research Pattern: Batch Pipeline

Commands + worktrees + merge = one repeatable machine for batch document work.



This exact machine is Exercise 2 — ten 10-K filings, in parallel, from one command.

the same shape — works for filings, transcripts, annual reports, interview notes — any corpus that splits into independent chunks

NUS BUSINESS SCHOOL

Quiz 2 — The Merge Came Back

Your parallel pipeline just finished — here is what's on your screen. What's your next move?

Agent: "Merged all 3 worktrees into main. The `pyproject.toml` conflict was resolved by keeping the formatting configuration. All done!"

```
$ ls out/ | wc -l
9   (you processed 10 filings)
```

What TWO problems do you see, and what is the ONE steering sentence you type next?

90 seconds with your neighbour — answer on the next slide, no peeking.

Quiz 2 — Answer

"Resolved by keeping one side" means the other side is gone — and 9 ≠ 10.

- **1** "kept the formatting configuration" = the **testing configuration was discarded** — keep-one masquerading as a resolution.
- **2** **9 outputs from 10 filings** — one filing vanished, and nothing logged it.

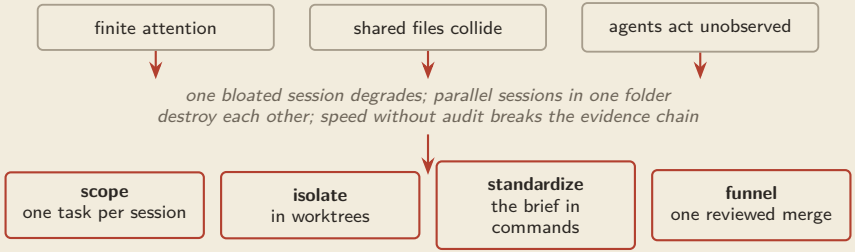
✓ "Re-examine the pyproject.toml merge — restore the testing config so BOTH sections survive; then find the missing 10th filing, tell me which group dropped it and why, and log it — never drop silently."

The merge is the audit point: count the outputs, read the resolution, trust neither claim.

same reflex as Quiz 1 — make every loss loud

Part 2 Conclusion: The Derivation Chain

Why the worktree ritual is necessary, not ceremony.



"One task, one worktree, one merge — and you read the merge."

this too is harness engineering: worktrees engineer the environment, commands capture the procedure

NUS BUSINESS SCHOOL

Exercise 2 — Watch the Launch

I run the launch once; you build the machine.

- 1 Show the `/run-pipeline` command file.
- 2 Invoke it on two small groups.
- 3 Watch worktrees appear (`git worktree list`).
- 4 Peek at one session working in its tree.

Your version starts from the command skeleton in the exercise folder.

Exercise 2 — Brief: A Parallel Pipeline

Ten 10-Ks: clean, extract, summarize — in parallel worktrees, from one command. **Goal:** process ~10 filings across isolated worktree sessions, launched by one custom command, then merge.

```
# .claude/commands/run-pipeline.md
For each filing group in $ARGUMENTS: add a worktree under .trees/,
run clean -> extract -> summarize, write results to out/, and log
any filing that fails --- never skip silently.
```

Stuck on where to start? Get ONE worktree running the three steps by hand first — then wrap it in the command.

NUS BUSINESS SCHOOL

Exercise 2 — Milestones

Four checkpoints — reach two and the pattern is yours.

- **1** Command file exists and invokes.
- **2** Worktrees appear on `git worktree list`; `.trees` ignored.
- **3** Each group's outputs land in `out/`.
- **4** Merge combines — with both halves of any conflict surviving.

Milestones 3–4 can be finished after class; the pattern is the point, not the finish line.

Exercise 2 — What Good Looks Like

Isolation held, and the merge lost nothing.

Good signs

- each group ran in its own worktree
- no session overwrote another's files
- outputs for all 10 filings — or a logged reason
- merge combined, not clobbered

How to verify

- `git worktree list` shows the trees
- count files in `out/` = 10 (or 10 − logged)
- diff one summary against its source filing
- confirm `.trees` is git-ignored

Parallelism is only a win if the merge is clean — check both halves.

Two parts down: you can direct one agent well, and many agents safely.

Part Three makes them yours.

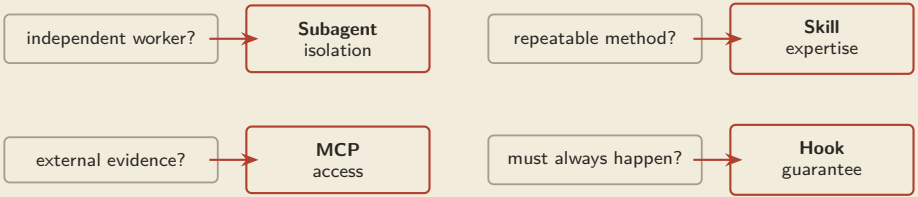
PART THREE

Personalization

- *Shape the agent to your research, not the reverse.*

What Is Your Research Workflow Missing?

Name the missing capability first; then choose the smallest lever that supplies it.

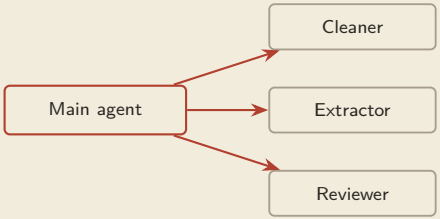


GitHub is the shared team surface; the four levers above change how the harness works.

the arc — Class 1: use the harness. Class 2, Part 3: rebuild it around your research

Subagents: A Team of Specialists

A subagent is a helper with its OWN context, OWN system prompt, OWN tools.



Each subagent has its **own**:

- context window
- system prompt
- tool set

The main agent delegates like a project manager — and can run them in parallel.

- The flow is exactly how you run an RA team: brief the specialist, let it work in its own space, receive a short report back.

own context — is the load-bearing part of the definition — why, on the next slide

Why Separate Context Matters

The specialist's mess stays in the specialist's office.

× **without subagents**

a deep dive (reading 30 files, failed attempts) floods the main window with noise → everything after degrades (the attention budget)

✓ **with a subagent**

the dive happens in the helper's own window; only a **short result summary** returns to the main thread

- The main agent stays lean while hard sub-problems get full attention.

subagents — attention-budget management, industrialized

Defining a Subagent

A Markdown file: name, when to use it, which tools, what it believes.

.claude/agents/extractor.md

name: extractor
description: use for pulling structured tables from filings
tools: read-only + file output

"You extract Item 1A risk tables… never modify source files… log every parse failure."

- Project-level `.claude/agents/` (shared via git) or user-level `~/.claude/agents/`.
- The *description* is how the main agent knows when to delegate.

Config format — verify vs current docs.

Worked Example: A Research Team

Cleaner, extractor, referee — and one non-negotiable rule: the author never reviews the author.

cleaner
normalizes raw filings; read raw, write only to `clean/`

extractor
pulls Item 1A tables; logs failures

referee
read-only tools; fresh context; "do not trust the author's summary" → PASS / PASS WITH LIMITATIONS / FAIL

Class 3A ships a full version of this referee.

independence — the reviewer must not share the author's context — a fresh window is what makes the audit independent

When NOT to Reach for a Subagent

Delegation has overhead — a simple lookup doesn't need a department.

Subagent	Just do it in-session
parallelizable work	single-file edits
needs isolation (deep dives, audits)	quick searches
separate workflows	sequential steps sharing context

Agents can over-delegate — a helper for what one search would answer. You're allowed to say "do it directly."

same judgement as hiring — not every task deserves a new specialist

How Do You Teach the Same Research Method Once?

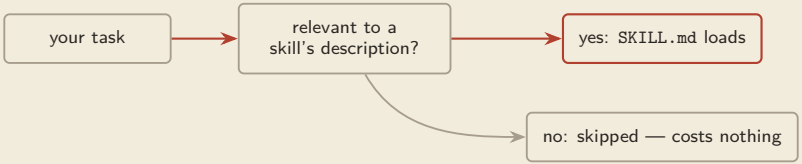
A skill packages your lab's method so any agent can apply it consistently.

The skill is the playbook; the agent is the worker using it.

where it lives — versioned in `.claude/skills/` — and it loads only for matching work

Progressive Disclosure

The specialist manual appears on the desk exactly when the job calls for it — and is invisible otherwise.



- Only lightweight descriptions are scanned up front; full instructions load on demand.
- Irrelevant expertise never touches the window — sessions stay lean.

Loading behaviour — verify vs current docs.

the attention budget again — expertise stored outside context, injected just-in-time

Anatomy of a `SKILL.md`

Description triggers it; body instructs it; resources equip it.

.claude/skills/cite-check/SKILL.md

name: cite-check
description: verify quotations and citations against source PDFs — use for any citation-checking task

1. locate the source · 2. match the quote *verbatim* · 3. check page numbers · 4. output a discrepancy table · **flag, never fix silently**

resources: `checklist.md`, `report-template.md`

the description is the trigger — write it like a "use when…" sentence, or the skill never fires

Where Knowledge Lives: Charter vs Skill

Needed every session → `CLAUDE.md`. Needed for one kind of task → skill.

`CLAUDE.md`	Skill
always loaded	loads on matching tasks only
short standing rules ("never edit raw")	long procedures welcome (the 12-step check)
pays attention-cost every session	costs nothing when irrelevant

The charter says WHAT is always true; skills say HOW we do things.

misplacing this — the #1 cause of bloated charters

Do You Need Access, Expertise, or Isolation?

Choose by the missing capability, not by the newest tool.

MCP	SKILL	SUBAGENT
connection standard	reusable playbook	separate worker
ACCESS	**EXPERTISE**	**ISOLATION**
live EDGAR, Zotero	your lab's method	deep dive or audit

A complex workflow may combine all three — but each solves a different problem.

Which Lever? A Decision Path

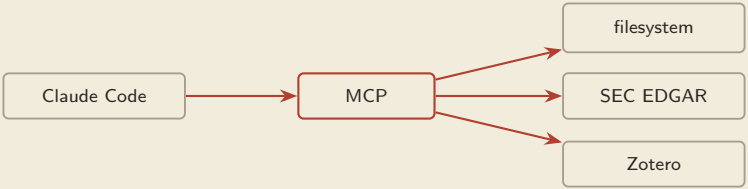
Ask what the problem is missing — the missing thing names the lever.

- Missing **data/system access** (live EDGAR, a database, a browser) → **MCP**.
- Missing **know-how** (a procedure done your way, repeatably) → **Skill**.
- Missing **isolation or parallelism** (deep dive, independent audit) → **Subagent**.
- Missing a **guarantee** (must ALWAYS/NEVER happen, no exceptions) → **Hook** — the lever we haven't met yet.

note the last row — instructions persuade, but only hooks guarantee

How Does the Agent Reach Evidence Outside the Project?

One protocol standard connects the agent to browsers, files, databases, APIs.



- MCP = Model Context Protocol — one standard plug for external systems.
- A connected server contributes new *tools* the agent can call in its loop. Manage with `/mcp`.
- Without a standard, every data source would need custom glue; with it, any server that speaks the protocol plugs into any agent that speaks it.

think device drivers — one standard, many devices — write once, plug in anywhere

Research Servers Worth Knowing

Connect the agent to where your evidence already lives.

- **filesystem** — controlled access to data folders beyond the project.
- **SEC EDGAR** — fetch filings by CIK/accession, live.
- **Zotero** — search your own reference library.

Examples of the category, not endorsements — start with the ONE server you actually need, and grow.

every server is new delegated power — connect deliberately, not maximally

Adding a Server

Name it, then give the command that launches it — after `--`.

```
claude mcp add playwright -- npx @playwright/mcp@latest
```

`playwright` = name you choose · `--` = separator · `npx @playwright/mcp@latest` = launch command

- List, inspect, and remove servers via `/mcp`.

Syntax and package names drift — verify vs current docs.

What Leaves Your Machine

An MCP server runs locally — but what the agent READS still goes to the model.

- The server process runs on your machine and mediates access.
- Content the agent retrieves through it **is sent to the model's API to reason over** — same as any file it reads.
- So a licensed-data question isn't "is MCP safe?" but **"does the license permit sending excerpts to a processor?"**

For restricted data (CRSP, Compustat, proprietary): institution policy first — same answer as Class 1B, same reason.

information boundary — know it before you widen it — Class 1A's information-set discipline

What If "Please Remember" Is Not Strong Enough?

For a non-negotiable rule, move control from the model into the harness.

hook = your shell command, bound to an event in the agent's loop; the harness executes it automatically — the model doesn't decide, can't skip it, can't forget it.

instructions & charters *persuade* (probabilistic) · hooks *execute* (deterministic)

the move — from "please remember to" to "this always happens"

Where Hooks Fire: The Lifecycle

Events along the loop you already know.

- Every hook is a pair: an *event* (when) + your *command* (what). The events cover the whole loop, so any moment you care about can carry a rule.

Also: `Notification`, `SubagentStop`, `PreCompact`, `SessionEnd` — manage with `/hooks`.

Event names and hook JSON schema — verify vs current docs.

the two workhorses — PreToolUse fires BEFORE an action — so it can block; PostToolUse fires after — so it can react

Hooks for Research Discipline

Turn this course's habits into rules the harness enforces.

Event	Action
`PreToolUse` on edits	**block any write to `data/raw/`** — the read-only zone, now physics
`PostToolUse` on `Write/Edit`	run the tests automatically — red-green without asking
`PreToolUse` on `git commit`	lint/format first
`PostToolUse` on `Bash`	**append every command to an audit log**
`Stop`	notify you

the audit-trail hook — answers Shirley's question — "could another researcher reconstruct the evidence path?" — with a file

The Charter Asks; the Hook Enforces

Same rule, two strengths — know which one your risk requires.

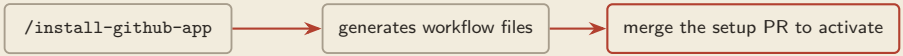
Where	Mechanism	Strength
`CLAUDE.md`: "never edit `data/raw/`"	read by the model, followed *almost* always	advisory
hook on `PreToolUse`: block writes to `data/raw/`	executed by the harness, followed **always**	guaranteed

Violation annoying → charter; violation unacceptable → hook.

Ideally both: the charter explains, the hook enforces.

The GitHub App

Install once; the agent then works inside issues and pull requests.



- `claude.yml` — fires when you mention `@claude` in an issue or PR.
- A review workflow auto-reviews new PRs (installer may name it `claude-code-review.yml` — check what yours generates).
- Runs in GitHub Actions with permissions you grant — review them at install; start on a test repo.

the same agent — now living where your coauthors already look

From Issue to Self-Reviewed PR

Describe a task in an issue; the agent opens a PR and reviews its own code.

- It is the same harness loop, running in the cloud: the issue is the prompt, the repo is the environment, the PR is the output.
- Asynchronous delegation: file the issue, get coffee, come back to a reviewed PR.

Self-Review Is a First Pass, Not a Verdict

The author reviewing the author re-runs the author's assumptions.

- The auto-review catches real things — style, obvious bugs, missed cases — accept the free labor.
- But it shares the author's blind spots — an independent check needs a *fresh context* (the referee pattern, and Sant'Anna's critic+fixer this morning).

The final judgement stays yours — automation bias doesn't get a GitHub exemption.

Your Toolkit Is Versioned Memory

Course message three, fully grown: everything you built today is files in git.

- `CLAUDE.md` — standing rules
- `.claude/commands/` — procedures
- `.claude/agents/` — specialists
- `.claude/skills/` — expertise
- hooks config — guarantees

one **versioned**, **shareable**, **compounding** research system

Class 3A takes this to its destination: the LLM-wiki — your whole research knowledge base, agent-readable.

the asymmetry — the agent forgets nightly; the repo remembers forever — that is the whole strategy

Concept Check: Four Levers, One Sentence Each

Before the final quiz applies them, retrieve the definitions — from memory.

In one sentence each: **what does a *skill* provide? a *subagent*? an *MCP server*? a *hook*?**

Bonus: which of the four is the only one the model *cannot* ignore?

Write four sentences — answer on the next slide, no peeking.

Concept Check — Answer

Four levers, four different missing things.

Skill	reusable **expertise** — your procedure, loaded on matching tasks
Subagent	**isolation** — a worker with its own context, prompt, and tools
MCP	**access** — a standard plug into external systems
Hook	a **guarantee** — deterministic and event-bound; the model can't skip it

access · expertise · isolation · guarantee — these four words decide Quiz 3.

Quiz 3 — Pick the Lever

Three real needs from a research team — assign the right lever to each.

(a) "Every 10-K table extraction must follow our lab's 12-step verification procedure — same steps, every time, any project."

(b) "The agent must NEVER write to `data/raw/` — a violation would corrupt the paper; 'almost never' is not acceptable."

(c) "We need filings fetched live from SEC EDGAR by accession number."

Options: `CLAUDE.md` rule · Skill · Subagent · MCP · Hook

60 seconds — one lever each; (b) is the one people get wrong — answer on the next slide, no peeking.

NUS BUSINESS SCHOOL

Quiz 3 — Answer

Access, expertise, isolation — and for guarantees, only a hook.

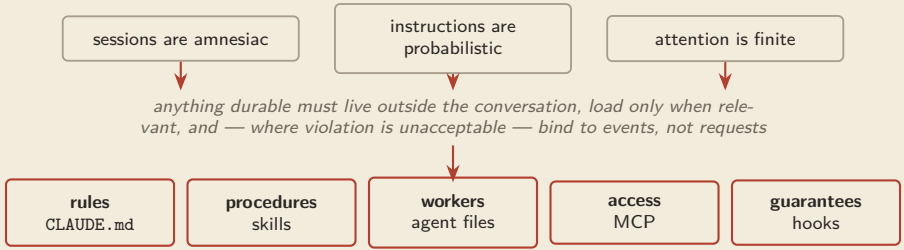
- **(a) Skill** — repeatable procedure, loads on matching tasks, portable across projects.
- **(c) MCP** — the missing thing is *access* to a live external system.
- **(b) Hook** (PreToolUse block) — the need says *NEVER*: a charter line is advisory — "almost never" is exactly what advisory means; a guarantee requires the deterministic lever.

knowledge → skill · access → MCP · promises → hooks

the tell — "must never" + a consequence you can't accept — stop persuading, start enforcing

Part 3 Conclusion: The Derivation Chain

Why personalization is engineering, not preference.



"Corrected it twice? Encode it once — knowledge in a file, promises in a hook."

all versioned in git — your harness engineering toolkit is complete: Sant'Anna's whole system is these five levers, grown up

Exercise 3 — Watch One Skill Get Born

I author the cite-check skill live; you author yours.

1. Create `.claude/skills/cite-check/SKILL.md`.
2. Write the "use when" description + 3-step body.
3. Trigger it with a citation task — watch it load.
4. Trigger an unrelated task — watch it stay silent.

The silence in step 4 is the passing test.

Exercise 3 — Brief: Make It Yours

One skill, one subagent, one MCP server — a toolkit you keep.

- 1 Write `.claude/skills/cite-check/SKILL.md` — verify quotations; flag, never fix silently.
- 2 Add `.claude/agents/extractor.md` — own prompt, restricted tools, logs failures.
- 3 Connect ONE MCP server — filesystem, EDGAR, or Zotero, whichever fits your work.

One working piece is a win — the skill is the easiest start; add the rest later.

NUS BUSINESS SCHOOL

Exercise 3 — Milestones

Checkpoints — in order of difficulty.

- **1** Skill file exists with a "use when" description.
- **2** Skill loads on-topic AND stays silent off-topic.
- **3** Subagent answers with its own restricted tool list.
- **4** `/mcp` lists your server; one live query returns.

Stuck? Recruit your neighbour's eyes — reading someone else's `SKILL.md` teaches both of you.

Exercise 3 — What Good Looks Like

Each piece loads at the right time and does one job well.

Good signs

- skill loads *only* on cite-check tasks
- subagent runs with its own tools
- `/mcp` lists the connected server
- the three cooperate on a real task

How to verify

- trigger an unrelated task — skill stays silent
- ask the subagent to report its tool set
- query the server for a single record
- spot-check that record by hand

Personalization earns its keep only if you can still verify every result.

Toolkit Show-and-Tell

Sixty seconds: what did you make it do?

- What lever did you build?
- What research task did it touch?
- What did you verify?

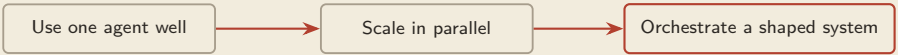
WRAP-UP

From Using to Orchestrating

- *The shift is in what you supervise.*

The Arc of the Day

Three stages, walked in one afternoon.



- Contracts, plan mode, memory, tests made *single* sessions reliable.
- Worktrees, commands, one audited merge made *many* sessions safe.
- Skills, subagents, MCP, hooks made the system *yours*.

NUS BUSINESS SCHOOL

Three Messages, Three Classes

The course's spine, one class from complete.

Message	Class 1B	**Class 2 (today)**	Class 3A
Agentic partner	brief one junior collaborator	orchestrate a team of them	give the team a library
Context engineering	one good prompt	scoped sessions, lean charters, just-in-time skills	a knowledge base engineered for agents
Persistent memory	`CLAUDE.md` exists	the full `.claude/` ecosystem, in git	the LLM-wiki

A Word on Guardrails

Speed without checking is a liability, not a feature.

Still your job, at any scale

- verify every result you'll rely on
- check citations and quotations
- review merged and generated code
- read the plans and the merges

Match tool to task

a regex or a word count needs no agent — and now, no *fleet* of agents either.

Systematic verification gets its own treatment in Class 3A.

Three Things to Remember

If you keep three ideas from today:

1

Contract and test.

Never type a task — type a contract; nothing claims "fixed" without red-then-green.

2

Scale with isolation.

One task, one worktree, one merge — and you read the merge.

3

Encode, don't repeat.

Corrected it twice? Knowledge in a file, promises in a hook.

These are the three Part-conclusion rules — one per part.

After Today

One practice task, one companion, one destination.

- **Practice this week** — run the parallel pipeline on your own documents (2–3 groups is plenty); or finish Exercise 2's milestones 3–4.
- **Study notes** — everything today, with every prompt, quiz answers, and the derivation chains: `Slides_Jiaqi/notes/study_notes_B.md`.
- **Next — Class 3A: Knowledge Management** — the `CLAUDE.md` ecosystem grows into an LLM-wiki; bring the toolkit you built today.

References


A. Ng and E. Schoppik. *Claude Code: A Highly Agentic Coding Assistant*. DeepLearning.AI × Anthropic short course, 2025.


Anthropic. *Claude Code Documentation* — Memory, Subagents, Skills, MCP, Hooks, GitHub Actions. `docs.claude.com/en/docs/claude-code`.


Anthropic. *Claude Code: Best Practices for Agentic Coding*. Engineering blog, 2025.


P. Sant'Anna. *My Claude Code Setup*. Public workflow guide, 2026. `psantanna.com`.


G. Chettiar. *Agent Harnesses: The Model Was Never the Bottleneck*. Blog essay.

The docs are the source of truth for exact syntax — that's why the slides kept saying verify.

CLASS 2

Thank you.

You now orchestrate, not just use.
Next: Class 3A — Knowledge Management.

BACKUP

Reference Material

- *For questions and deeper dives.*

Backup: The Full Contract Template

The five-part contract scaled to a real refactor.

```
Context: @backend/rag_system.py mixes retrieval and generation.
Goal: split into a retriever module and a generator module.
Constraints: keep the public interface; add tests for each.
Verification: all existing tests still pass; each new module has
a failing-then-passing test.
Method: brainstorm two plans with parallel subagents first.
Think hard.
```

Same skeleton as the contract-recap slide — the shape scales from one-liner to full refactor.

It composes three lessons: the contract, the subagent brainstorm, and the thinking ladder.

Backup: Command Cheat-Sheet

Everything typed today, one table.

Command	Does
`Shift+Tab`	toggle Plan Mode
`Esc`	interrupt
`/clear`	wipe context
`/compact`	compress, keep summary
`/init`	draft `CLAUDE.md`
`#`	save a memory

Command	Does
`@`	point at a file
`/mcp`	manage servers
`/agents`	manage subagents
`/hooks`	manage hooks
`/install-github-app`	GitHub workflows
`git worktree add|list|remove`	parallel workspaces

Install command and package name — verify vs current docs. Also in the study notes — no transcribing.

Backup: Hook Schema Caveats

The conceptual shape — match a tool, run a command. Do not invent fields.

```
{ "matcher": "Write",
  "hooks": [ { "type": "command",
        "command": "pytest -q" } ] }
```

- Use official event names — `PreToolUse`, `PostToolUse`, `Stop`, …
- This is the idea, not the spec.

The exact hook JSON schema changes — verify vs current docs before relying on it.

Backup: Worktree Troubleshooting

The three worktree stumbles and their fixes.

- `add` fails on a dirty tree → commit or stash first.
- Leftover trees after merging → `git worktree remove <path>` then `git branch -d <name>`; or ask the agent, then confirm with `git worktree list`.
- `@`-completion feels off inside a tree → worktrees run outside the main folder; prefer explicit paths in prompts.

Keep `.trees` in `.gitignore` permanently.

Backup: Cost & Capacity

How many parallel agents is reasonable?

- Each session is an independent consumer of compute and budget — `/cost` shows a session's spend.
- Laptops handle a handful of sessions comfortably; the constraint is usually your attention and budget, not the CPU.
- Scale like an empiricist: 2–3 sessions, measure, then grow.

the real bottleneck — the merge audit — don't launch more work than you can review

Backup: MCP Under the Hood

What actually happens when you add a server.

- The launch command starts a local *server process*; the harness speaks the protocol to it.
- The server *advertises tools*; those tools join the agent's toolset like any other.
- Remove it and the tools vanish — access is exactly as wide as your server list.

audit your `/mcp` list — the way you'd audit RA data access — it IS an access list

Backup: Glossary

Eleven terms from today, one line each.

harness	the software frame giving the model tools, memory, and a loop
harness engineering	improving the system by engineering everything except the weights
context window	everything the model sees right now
attention dilution	more instructions → weaker adherence to each
contract	context / goal / example / constraints / verification
worktree	extra working folder sharing one git history
subagent	helper with its own context, prompt, tools
skill	`SKILL.md` expertise, loaded on demand
progressive disclosure	full instructions load only when relevant
MCP	protocol standard connecting external systems
hook	shell command bound to a lifecycle event — deterministic